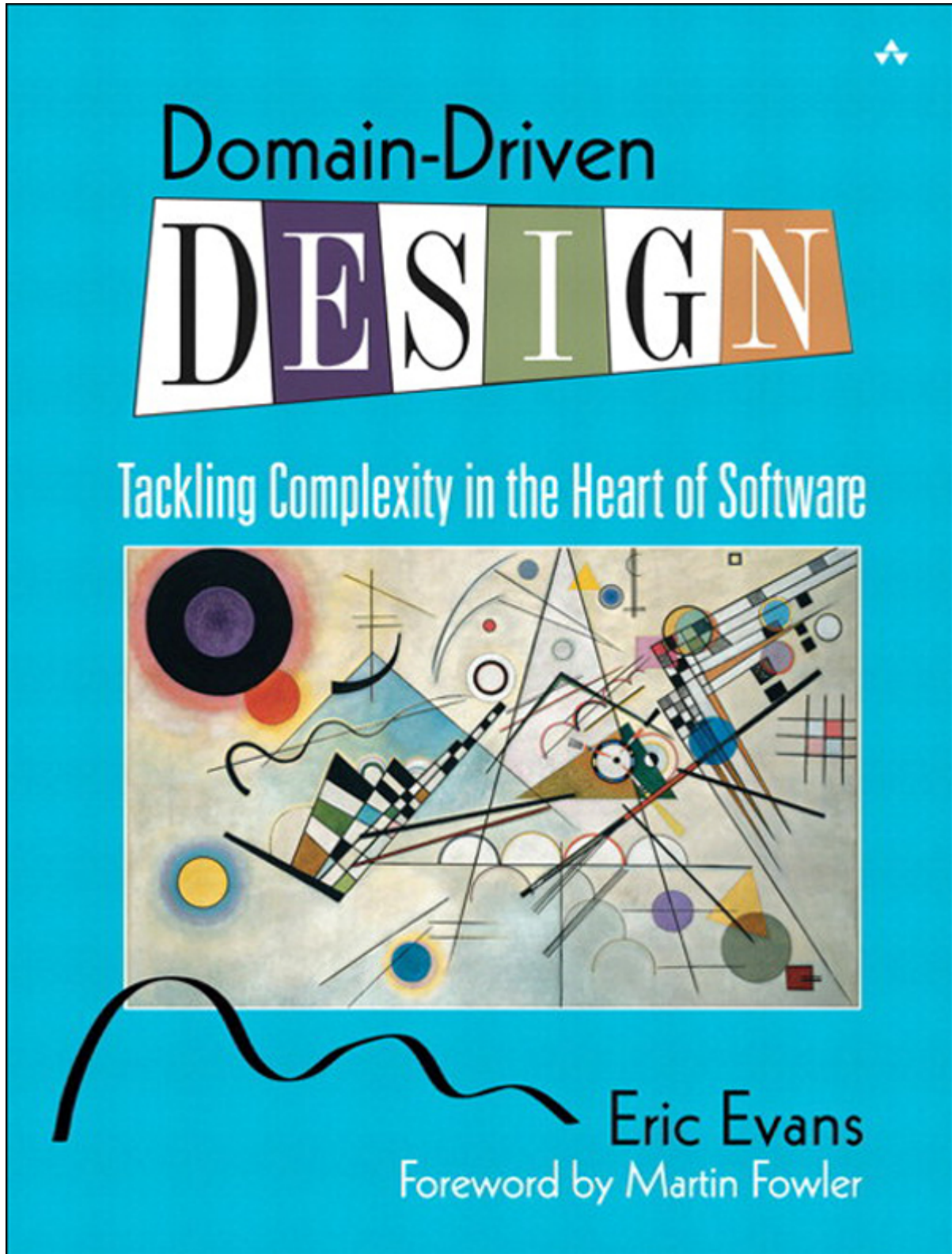
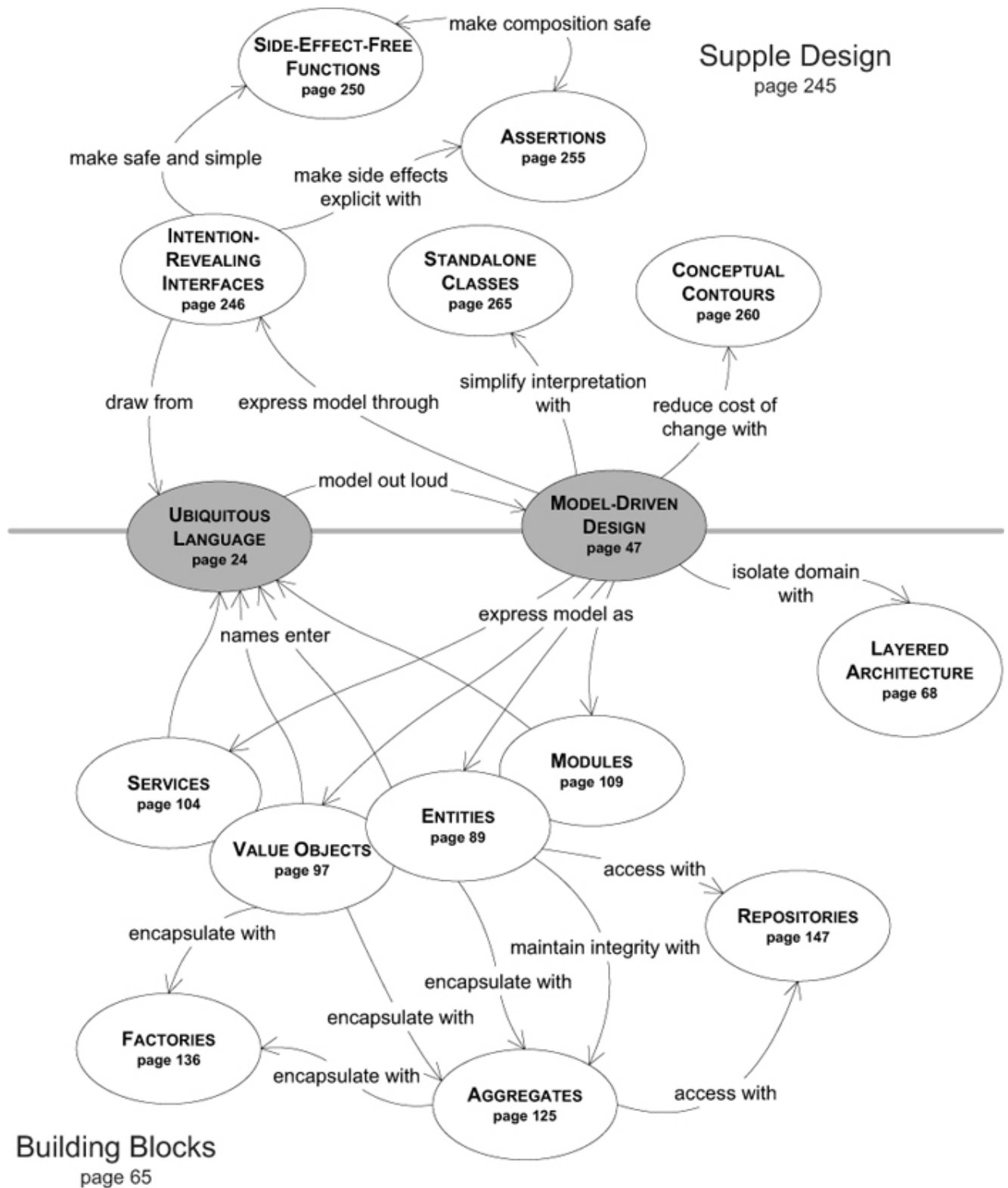
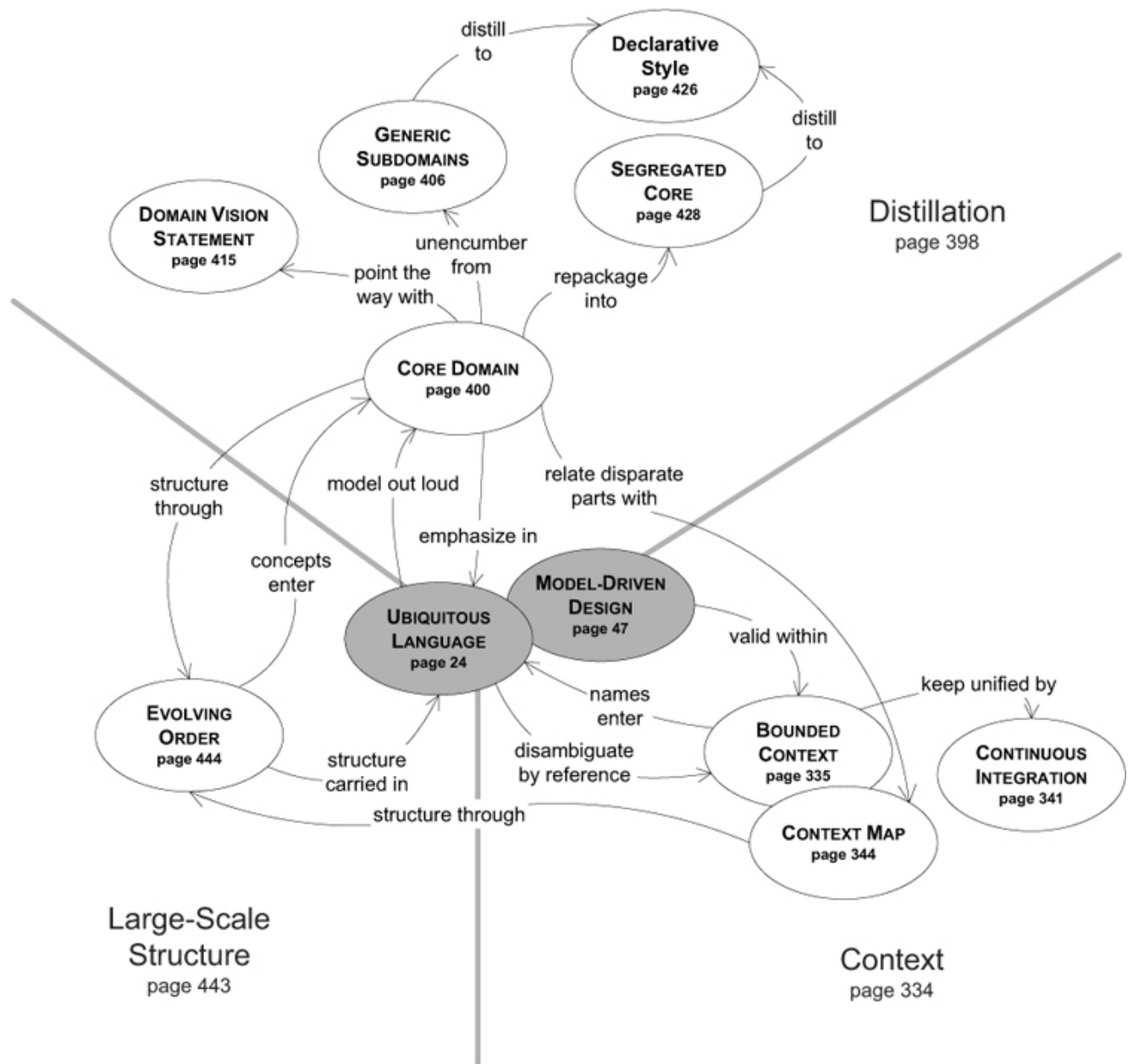




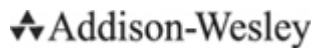




Domain-Driven Design

Tackling Complexity in the Heart of Software

Eric Evans



Upper Saddle River, NJ • Boston • Indianapolis • San Francisco
New York • Toronto • Montreal • London • Munich • Paris • Madrid
Capetown • Sydney • Tokyo • Singapore • Mexico City

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and Addison-Wesley was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

See page [517](#) for photo credits.

The publisher offers discounts on this book when ordered in quantity for bulk purchases and special sales. For more information, please contact:

U.S. Corporate and Government Sales
(800) 382-3419
corpsales@pearsontechgroup.com

For sales outside of the U.S., please contact:

International Sales
international@pearsoned.com

Visit Addison-Wesley on the Web: www.awprofessional.com

Library of Congress Cataloging-in-Publication Data

Evans, Eric, 1962–

Domain-driven design : tackling complexity in the heart of software / Eric
Evans.

p. cm.

Includes bibliographical references and index.

ISBN 0-321-12521-5

1. Computer software—Development. 2. Object-oriented programming
(Computer science) I. Title.

QA76.76.D47E82 2003
005.1—dc21

2003050331

Copyright © 2004 by Eric Evans

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form, or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior consent of the publisher. Printed in the United States of America. Published simultaneously in Canada.

For information on obtaining permission for use of material from this work, please submit a written request to:

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Pearson Education, Inc.
Rights and Contracts Department
500 Boylston Street, Suite 900
Boston, MA 02116
Fax: (617) 671-3447

ISBN 0-321-12521-5

Text printed in the United States on recycled paper at Courier in Westford,
Massachusetts.

Twenty-First printing, July 2015

Praise for *Domain-Driven Design*

“This book belongs on the shelf of every thoughtful software developer.”

—Kent Beck

“Eric Evans has written a fantastic book on how you can make the design of your software match your mental model of the problem domain you are addressing.

“His book is very compatible with XP. It is not about drawing pictures of a domain; it is about how you think of it, the language you use to talk about it, and how you organize your software to reflect your improving understanding of it. Eric thinks that learning about your problem domain is as likely to happen at the end of your project as at the beginning, and so refactoring is a big part of his technique.

“The book is a fun read. Eric has lots of interesting stories, and he has a way with words. I see this book as essential reading for software developers—it is a future classic.”

—Ralph Johnson, author of *Design Patterns*

“If you don’t think you are getting value from your investment in object-oriented programming, this book will tell you what you’ve forgotten to do.”

—Ward Cunningham

“What Eric has managed to capture is a part of the design process that experienced object designers have always used, but that we have been singularly unsuccessful as a group in conveying to the rest of the industry. We’ve given away bits and pieces of this knowledge . . . but we’ve never organized and systematized the principles of building domain logic. This book is important.”

—Kyle Brown, author of *Enterprise Java Programming with IBM WebSphere*

“Eric Evans convincingly argues for the importance of domain modeling as the central focus of development and provides a solid framework and set of techniques for accomplishing it. This is timeless wisdom, and will hold up long after the methodologies *du jour* have gone out of fashion.”

—Dave Collins, author of *Designing Object-Oriented User Interfaces*

“Eric weaves real-world experience modeling—and building—business applications into a practical, useful book. Written from the perspective of a trusted practitioner, Eric’s descriptions of ubiquitous language, the benefits of sharing models with users, object life-cycle management, logical and physical application structuring, and the process and results of deep refactoring are major contributions to our field.”

—Luke Hohmann, author of *Beyond Software Architecture*

Contents

[*Foreword*](#)

[*Preface*](#)

[*Acknowledgments*](#)

Part I Putting the Domain Model to Work

[*Chapter 1: Crunching Knowledge*](#)

[*Ingredients of Effective Modeling*](#)

[*Knowledge Crunching*](#)

[*Continuous Learning*](#)

[*Knowledge-Rich Design*](#)

[*Deep Models*](#)

[*Chapter 2: Communication and the Use of Language*](#)

[*UBIQUITOUS LANGUAGE*](#)

[*Modeling Out Loud*](#)

[*One Team, One Language*](#)

[*Documents and Diagrams*](#)

[*Written Design Documents*](#)

[*Executable Bedrock*](#)

[*Explanatory Models*](#)

[*Chapter 3: Binding Model and Implementation*](#)

[*MODEL-DRIVEN DESIGN*](#)

[*Modeling Paradigms and Tool Support*](#)

[*Letting the Bones Show: Why Models Matter to Users*](#)

[*HANDS-ON MODELERS*](#)

Part II The Building Blocks of a Model-Driven Design

[*Chapter 4: Isolating the Domain*](#)

[*LAYERED ARCHITECTURE*](#)

[*Relating the Layers*](#)

[*Architectural Frameworks*](#)

[*The Domain Layer Is Where the Model Lives*](#)

[*THE SMART UI “ANTI-PATTERN”*](#)

Other Kinds of Isolation

Chapter 5: A Model Expressed in Software

Associations

ENTITIES (A.K.A. REFERENCE OBJECTS)

Modeling ENTITIES

Designing the Identity Operation

VALUE OBJECTS

Designing VALUE OBJECTS

Designing Associations That Involve VALUE OBJECTS

SERVICES

SERVICES and the Isolated Domain Layer

Granularity

Access to SERVICES

MODULES (A.K.A. PACKAGES)

Agile MODULES

The Pitfalls of Infrastructure-Driven Packaging

Modeling Paradigms

Why the Object Paradigm Predominates

Nonobjects in an Object World

Sticking with MODEL-DRIVEN DESIGN When Mixing Paradigms

Chapter 6: The Life Cycle of a Domain Object

AGGREGATES

FACTORIES

Choosing FACTORIES and Their Sites

When a Constructor Is All You Need

Designing the Interface

Where Does Invariant Logic Go?

ENTITY FACTORIES Versus VALUE OBJECT FACTORIES

Reconstituting Stored Objects

REPOSITORIES

Querying a REPOSITORY

Client Code Ignores REPOSITORY Implementation; Developers Do Not

Implementing a REPOSITORY

Working Within Your Frameworks

The Relationship with FACTORIES

Designing Objects for Relational Databases

Chapter 7: Using the Language: An Extended Example

Introducing the Cargo Shipping System

Isolating the Domain: Introducing the Applications

Distinguishing ENTITIES and VALUE OBJECTS

Role and Other Attributes

Designing Associations in the Shipping Domain

AGGREGATE Boundaries

Selecting REPOSITORIES

Walking Through Scenarios

*Sample Application Feature: Changing the Destination of a **Cargo***

Sample Application Feature: Repeat Business

Object Creation

*FACTORIES and Constructors for **Cargo***

*Adding a **Handling Event***

Pause for Refactoring: An Alternative Design of the **Cargo** AGGREGATE

MODULES in the Shipping Model

Introducing a New Feature: Allocation Checking

Connecting the Two Systems

Enhancing the Model: Segmenting the Business

Performance Tuning

A Final Look

Part III Refactoring Toward Deeper Insight

Chapter 8: Breakthrough

Story of a Breakthrough

A Decent Model, and Yet . . .

The Breakthrough

A Deeper Model

A Sobering Decision

The Payoff

Opportunities

Focus on Basics

Epilogue: A Cascade of New Insights

Chapter 9: Making Implicit Concepts Explicit

Digging Out Concepts

Listen to Language

Scrutinize Awkwardness
Contemplate Contradictions
Read the Book
Try, Try Again

How to Model Less Obvious Kinds of Concepts

Explicit Constraints
Processes as Domain Objects
SPECIFICATION
Applying and Implementing SPECIFICATION

Chapter 10: *Supple Design*

INTENTION-REVEALING INTERFACES
SIDE-EFFECT-FREE FUNCTIONS
ASSERTIONS
CONCEPTUAL CONTOURS
STANDALONE CLASSES
CLOSURE OF OPERATIONS

Declarative Design

Domain-Specific Languages
A Declarative Style of Design
Extending SPECIFICATIONS in a Declarative Style
Angles of Attack
Carve Off Subdomains
Draw on Established Formalisms, When You Can

Chapter 11: *Applying Analysis Patterns*

Chapter 12: *Relating Design Patterns to the Model*

STRATEGY (A.K.A. POLICY)
COMPOSITE
Why Not FLYWEIGHT?

Chapter 13: *Refactoring Toward Deeper Insight*

Initiation
Exploration Teams
Prior Art
A Design for Developers
Timing
Crisis as Opportunity

Part IV Strategic Design

Chapter 14: Maintaining Model Integrity

BOUNDED CONTEXT

Recognizing Splinters Within a BOUNDED CONTEXT

CONTINUOUS INTEGRATION

CONTEXT MAP

Testing at the CONTEXT Boundaries

Organizing and Documenting CONTEXT MAPS

Relationships Between BOUNDED CONTEXTS

SHARED KERNEL

CUSTOMER/SUPPLIER DEVELOPMENT TEAMS

CONFORMIST

ANTICORRUPTION LAYER

Designing the Interface of the ANTICORRUPTION LAYER

Implementing the ANTICORRUPTION LAYER

A Cautionary Tale

SEPARATE WAYS

OPEN HOST SERVICE

PUBLISHED LANGUAGE

Unifying an Elephant

Choosing Your Model Context Strategy

Team Decision or Higher

Putting Ourselves in Context

Transforming Boundaries

This document was truncated here because it was created in the Evaluation Mode.